

## Bloque 3 - Tema 3

---

LENGUAJES DE PROGRAMACIÓN.  
REPRESENTACIÓN DE TIPOS DE DATOS.  
OPERADORES. INSTRUCCIONES CONDICIONALES.  
BUCLES Y RECURSIVIDAD. PROCEDIMIENTOS,  
FUNCIONES Y PARÁMETROS. VECTORES Y  
REGISTROS. ESTRUCTURA DE UN PROGRAMA

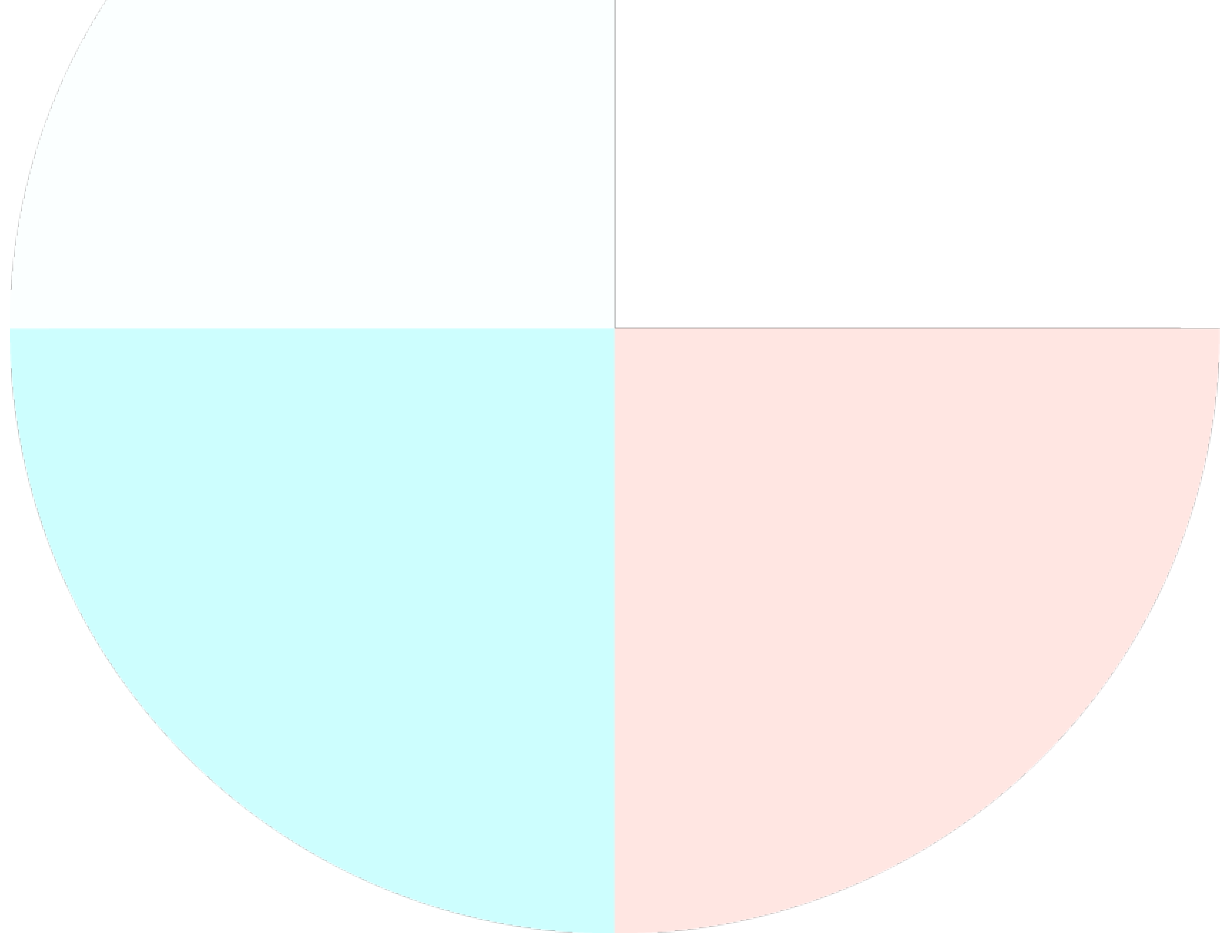
---

**PREPARACIÓN OPOSICIONES**

**TÉCNICOS AUXILIARES DE INFORMÁTICA**

# ÍNDICE

|                                                        |    |
|--------------------------------------------------------|----|
| ÍNDICE .....                                           | 2  |
| 1. LENGUAJES DE PROGRAMACIÓN .....                     | 3  |
| 1. Definiciones básicas .....                          | 3  |
| 2. Compiladores .....                                  | 4  |
| 3. Intérpretes.....                                    | 6  |
| 4. Clasificación de los lenguajes de programación..... | 6  |
| 5. Principales lenguajes de programación.....          | 18 |
| 2. REPRESENTACIÓN DE TIPOS DE DATOS .....              | 20 |
| 3. OPERADORES .....                                    | 23 |
| 4. INSTRUCCIONES CONDICIONALES.....                    | 27 |
| 5. BUCLES .....                                        | 29 |
| 6. PROCEDIMIENTOS, FUNCIONES Y PARÁMETROS.....         | 32 |
| 7. VECTORES Y REGISTROS .....                          | 35 |
| 8. ESTRUCTURA DE UN PROGRAMA.....                      | 37 |



# 1. LENGUAJES DE PROGRAMACIÓN

## 1. Definiciones básicas

**Algoritmo:** es un conjunto de acciones o secuencia de operaciones que ejecutadas en un determinado orden resuelven un problema concreto. Existen n algoritmos para resolver un mismo problema, escoger el más eficiente.

Representa una secuencia ordenada de pasos (sin ambigüedades), repetible, que da solución a un determinado problema.

Las características fundamentales que debe cumplir todo algoritmo son:

- Debe ser preciso e indicar el orden de realización de cada paso.
- Debe estar definido (si se repiten n veces los pasos se debe obtener siempre el mismo resultado).
- Debe ser finito (debe tener un número finito de pasos).
- Es independiente del lenguaje de programación que se utilice.
- La definición de un algoritmo debe describir 3 partes: Entrada, Proceso, Salida.
- La programación es adaptar el algoritmo al ordenador.
- El algoritmo es independiente de donde se implemente (hardware o software).

**Instrucción:** cada una de las tareas elementales a realizar por un elemento de computación.

**Programa:** es un conjunto de instrucciones que al ser ejecutadas resuelven un problema.

Un programa tiene 3 partes:

1. **Entrada de datos:** normalmente se va a ejecutar a través de instrucciones de lectura, y en lo que se le pide al usuario la información que el programa va a necesitar para ejecutarse, o bien se leen los datos desde un sistema de almacenamiento secundario.
2. **Acciones de un algoritmo:** parte en la que se resuelve el problema usando los datos de entrada. En la parte de las acciones a ejecutar se distinguirán dos partes:
  - Declaración de variables.
  - Instrucciones del programa.
3. **Salida:** mostrar en un dispositivo de salida los resultados de las acciones anteriormente realizadas o grabarlos en un sistema de almacenamiento secundario. Son acciones de escritura.

**Lenguaje de programación:** conjunto de normas «lingüísticas» que permiten escribir un programa y que éste sea entendido por el ordenador y pueda ser trasladado a ordenadores «similares» para su funcionamiento en otros sistemas.

**Subprograma:** fragmento de programa que resuelve un subproblema con entidad propia.

Los subprogramas dividen las tareas grandes de computación en varias más pequeñas y especializadas:

- Permiten la reusabilidad.
- Distribuyen la programación: en distintos niveles de abstracción o en el desarrollo por un equipo de personas.

**Función:** subprograma que se caracteriza por disponer de «n» entradas, pero únicamente de una salida (siempre una).

**Procedimiento:** subprograma que se caracteriza por disponer de «n» entradas y «n» salidas, siendo  $n = \{0, 2, \dots n\}$ .

Un **traductor** es un programa que procesa un texto fuente y genera un texto objeto. Según la forma de la traducción, los traductores se pueden clasificar en compiladores e intérpretes.

## 2. Compiladores

Un **COMPILADOR** es un programa informático que traduce o transforma un programa escrito en un lenguaje de programación (código fuente) a su equivalente en otro lenguaje de programación de más bajo nivel (código objeto), que la máquina será capaz de interpretar. Usualmente el segundo lenguaje es lenguaje de máquina, pero también puede ser un código intermedio (bytecode), o simplemente texto.

Este proceso de traducción se conoce como **compilación**.

Generalmente estas fases se agrupan en dos tareas: el análisis del programa fuente y la síntesis del programa objeto.

- **Análisis:** se trata de la comprobación de la corrección del programa fuente, e incluye las fases:
  - **Análisis léxico (scanner):** consiste en la descomposición del programa fuente en componentes léxicos (tokens = secuencia de caracteres con significado propio).
  - **Análisis sintáctico (parser):** agrupación jerárquica de los componentes léxicos en frases gramaticales en un árbol para comprobar si es sintácticamente correcto, esto es, si sus estructuras (expresiones, sentencias y asignaciones) están construidas según las reglas del lenguaje.
  - **Análisis semántico:** comprobación de la validez semántica de las sentencias aceptadas en la fase de Análisis Sintáctico (operandos permitidos para un operador, variables usadas y no definidas, llamadas a procedimientos con número y/o tipo de parámetros incorrectos).

- **Síntesis:** su objetivo es la generación de la salida expresada en el lenguaje objeto y suele estar formado por una o varias combinaciones de fases de:
  - Generación de código: normalmente se trata de generar código intermedio o de código objeto y
  - Optimización de código: en las que se busca obtener un código lo más eficiente posible.

El proceso de compilación involucra **4 etapas** sucesivas: preprocesamiento, compilación, ensamblado y enlazado. Para pasar de un programa fuente escrito por un humano a un archivo ejecutable es necesario realizar estas cuatro etapas en forma sucesiva:

- (1) **Preprocesado:** en esta etapa se interpretan las directivas. Entre otras cosas, las variables inicializadas con `#define` son sustituidas en el código por su valor en todos los lugares donde aparece su nombre.
- (2) **Compilación:** la compilación transforma el código en el lenguaje ensamblador propio del procesador de nuestra máquina.
- (3) **Ensamblado:** el ensamblado transforma el programa escrito en lenguaje ensamblador a código objeto, un archivo binario en lenguaje de máquina ejecutable por el procesador.
- (4) **Enlazado:** para hacer referencia a las funciones incluidas en nuestro código que se encuentran ya compiladas y ensambladas en bibliotecas existentes en el sistema. Es preciso incorporar de algún modo el código binario de estas funciones a nuestro ejecutable. En esto consiste la etapa de enlace, donde se reúnen uno o más módulos en código objeto con el código existente en las bibliotecas.

Existen 2 modos de realizar el enlace:

- Estático: los binarios de las funciones se incorporan al código binario de nuestro ejecutable.
- Dinámico: el código de las funciones permanece en la biblioteca; nuestro ejecutable cargará en memoria la biblioteca y ejecutará la parte de código correspondiente en el momento de correr el programa.

El **enlazado dinámico** permite crear un ejecutable de menor tamaño, pero requiere tener disponible el acceso a las bibliotecas cuando el programa esté en ejecución. El **enlazado estático** crea un programa autónomo, pero al precio de aumentar el tamaño del ejecutable binario.

Variante: un **compilador cruzado** (cross compiler) es un compilador capaz de crear código ejecutable para otra plataforma distinta a aquella en la que se ejecuta.

Variante: un **transpilador** es un compilador que traduce un código en un lenguaje fuente a otro lenguaje fuente (distinto del primero, pero al mismo nivel de abstracción). Por ejemplo, de TypeScript a JavaScript.

### 3. Intérpretes

Un **INTÉRPRETE** es un programa informático que ejecuta las instrucciones escritas en un lenguaje de programación dado (código fuente), sin necesidad de generar un programa objeto.

Los intérpretes se diferencian de los compiladores en que mientras estos traducen un programa desde su descripción en un lenguaje de programación al código de máquina del sistema, los intérpretes sólo realizan la traducción a medida que sea necesaria, típicamente, instrucción por instrucción, y normalmente no guardan el resultado de dicha traducción.

La interpretación de código es más lenta que la ejecución de código compilado porque el intérprete debe analizar cada sentencia en el programa cada vez que es ejecutada y entonces realizar la acción deseada, mientras que el código compilado solo realiza la acción dentro de un determinado contexto fijo por la compilación. Este análisis en tiempo de ejecución se conoce como "sobrecarga interpretativa".

### 4. Clasificación de los lenguajes de programación

Los lenguajes de programación se pueden clasificar según varios criterios, entre los que se encuentran: nivel de abstracción, propósito, evolución histórica, manera de ejecutarse, manera de abordar la tarea a realizar, paradigma de programación, lugar de ejecución, concurrencia, interactividad, realización visual, determinismo, grado de tipado, clase de tipado y utilidad.

Hay que tener en cuenta también que, en la práctica, la mayoría de los lenguajes no pueden ser puramente clasificados en una categoría, pues surgen incorporando ideas de otros lenguajes y de otras filosofías de programación, pero no importa al establecer las clasificaciones, pues el auténtico objetivo de las mismas es mostrar los rangos, las posibilidades y tipos de lenguajes que hay.

#### (1) Según el nivel de ABSTRACCIÓN

Según el nivel de abstracción, o sea, según el grado de cercanía a la máquina tenemos:

- **Lenguaje MÁQUINA:** las instrucciones se representan mediante combinaciones de ceros y unos (código binario). Por lo tanto, es directamente ejecutable por el microprocesador de un computador. Este lenguaje está compuesto por un conjunto de instrucciones que determinan acciones al ser tomadas por la máquina.
- **Lenguajes de BAJO nivel:** son lenguajes que asignan a cada instrucción del lenguaje máquina un nombre nemotécnico con el fin de facilitar su uso y significado: lenguajes simbólicos. Apenas existe abstracción del microprocesador. El lenguaje ensamblador es el propio de este nivel.

- **Lenguajes de MEDIO nivel:** tienen ciertas características que los acercan a los lenguajes de bajo nivel pero teniendo, al mismo tiempo, ciertas cualidades que lo hacen un lenguaje más cercano al humano y, por tanto, de alto nivel.
- **Lenguajes de ALTO nivel:** se trata de lenguajes próximos al lenguaje natural que proporcionan instrucciones para representar la lógica del programa y de estructuras de datos independientes de la arquitectura hardware subyacente.

Las ventajas de este tipo de lenguajes son la transportabilidad entre plataformas, la independencia de la arquitectura hardware concreta y la facilidad de programación y mantenimiento.

Entre los inconvenientes destacan el no aprovechamiento de la arquitectura interna y el aumento del tiempo de compilación y ejecución.

## (2) Según el PROPÓSITO

Según el propósito, es decir, el tipo de problemas a resolver para el que se crearon los lenguajes, tenemos:

- **Lenguajes de propósito GENERAL:** aptos para todo tipo de tareas.  
Lenguajes: C, C++, C#, Java, Visual Basic.
- **Lenguajes de propósito ESPECÍFICO:** desarrollados para un objetivo muy concreto.
  - Gestión de empresas: COBOL.
  - Cálculo científico: Fortran, M.
  - Educación: Scratch, Python, Processing, Java, Lua, JavaScript, Racket, Raptor, Pascal, Oz, Etoys, ABC, Grace, Logo.
  - Gestión de bases de datos: SQL, ABAP.
  - Inteligencia artificial: Prolog, Lisp, Haskell, R, F#.
  - Web: PHP, Perl, Python, Javascript, Java.
- **Lenguajes de programación de SISTEMAS:** diseñados para realizar sistemas operativos o drivers.  
Lenguajes: C.
- **Lenguajes de SCRIPT:** para realizar tareas varias de control y auxiliares. Antiguamente eran los llamados lenguajes de procesamiento por lotes (batch) o JCL ("Job Control Languages").  
Lenguajes: Bourne Shell (sh), Bourne-Again bash (bash), Debian Almquist Shell (dash), Korn Shell (ksh), Z Shell (zsh), C Shell (csh), TENEX C Shell (tcsh).

### (3) Según el PARADIGMA de programación

Un **paradigma de programación** indica un método de realizar cálculos y la manera en que se deben estructurar y organizar las tareas que debe llevar a cabo un programa. Los paradigmas fundamentales están basados en diferentes modelos de cómputo y por lo tanto afectan a las construcciones más básicas de un programa.

El paradigma de programación es el estilo de programación empleado. Algunos lenguajes soportan varios paradigmas, y otros sólo uno. Por ejemplo, C++ puede ser empleado para desarrollar software utilizando para ello un modelo de programación puramente orientado a objetos o bien puramente estructurado. En casos como el de C++, somos los programadores los que decidimos que utilizar. A este tipo de lenguajes se les conoce como multiparadigma. Por otro lado, algunos lenguajes han sido diseñados para soportar un único paradigma de programación, ese es el caso de Smalltalk que soporta únicamente la programación orientada a objetos o Haskell que solo soporta la programación funcional.

Se puede decir que históricamente han ido apareciendo para facilitar la tarea de programar según el tipo de problema a abordar, o para facilitar el mantenimiento del software, o por otra cuestión similar, por lo que todos corresponden a lenguajes de alto nivel, estando los lenguajes ensambladores “atados” a la arquitectura de su procesador correspondiente. Paradigmas:

|                         |                         |
|-------------------------|-------------------------|
| - Imperativo.           | - Dirigida por eventos. |
| - Orientado a objetos.  | - Por restricciones.    |
| - Funcional.            | - Reactiva.             |
| - Lógico.               | - Reflexivo.            |
| - Declarativo.          | - Concurrente.          |
| - Orientado a aspectos. |                         |

Los principales son:

- **Paradigma IMPERATIVO o por procedimientos:** describe **cómo** debe realizarse el cálculo, no el por qué. Un cómputo consiste en una serie de sentencias, ejecutadas según un control de flujo **explícito**, que **modifican el estado** del programa, es decir, divide el problema en partes más pequeñas, que serán realizadas por subprogramas (subrutinas, funciones, procedimientos), que se llaman unas a otras para ser ejecutadas.

Los programas escritos bajo este paradigma son secuenciales, es decir, se indican los pasos o tareas que deben realizar siguiendo estas reglas:

- El programa tiene un diseño modular.
- Los módulos son diseñados de manera que un problema complejo se divide en problemas más simples.
- Cada módulo se codifica utilizando las tres estructuras de control básicas: secuencia, selección y repetición.

Lenguajes: C, Pascal, BASIC, ADA, COBOL, FORTRAN.

- **Programación ORIENTADOS A OBJETOS:** también denominada POO, usa los objetos en sus interacciones, para diseñar aplicaciones y programas informáticos. Está basado en varias técnicas, incluyendo herencia, cohesión, abstracción, polimorfismo, acoplamiento y encapsulamiento.

Así, este paradigma se base en el concepto de objeto y de clases de objetos. Un objeto es una variable equipada con un conjunto de operaciones que le pertenecen o están definidas para ellos.

Las clases son las "plantillas" desde las que se pueden crear o instanciar múltiples objetos del mismo tipo. O lo que es lo mismo, representan la abstracción de las características comunes de un tipo determinado de objetos. Los componentes de las clases son:

- Atributos: elementos que mantienen el estado del objeto.
- Métodos: mensaje para realizar alguna acción en un objeto.

Los objetos son entidades que tienen un determinado estado, comportamiento (método) e identidad:

- El **ESTADO** está compuesto de datos o informaciones.
- El **COMPORTAMIENTO** está definido por los métodos o mensajes a los que sabe responder.
- La **IDENTIDAD** es una propiedad de un objeto que lo diferencia del resto.

Las características más importantes de la programación orientada a objetos son las siguientes:

- Abstracción: denota las características esenciales de un objeto, donde se captura su comportamiento.
- Encapsulamiento: hace referencia a la reunión de todos los elementos que pueden considerarse pertenecientes a una misma entidad. Ello supone el aumento de la cohesión de los componentes. No confundir con el principio de ocultación.
- Principio de ocultación: la estructura interna permanece oculta, tanto para el usuario como para otros objetos diferentes. La información contenida en el objeto será accesible sólo a través de la ejecución de los métodos adecuados creándose una interfaz para la comunicación con el mundo exterior.
- Polimorfismo: comportamientos diferentes, asociados a objetos distintos, pueden compartir el mismo nombre; al llamarlos por ese nombre se utilizará el comportamiento correspondiente al objeto que se esté usando. Dicho de otro modo, las referencias y las colecciones de objetos pueden contener objetos de diferentes tipos y la invocación de un comportamiento en una referencia producirá el comportamiento correcto para el tipo real del objeto referenciado. Cuando esto ocurre en "tiempo de ejecución", se llama "asignación tardía" o "asignación dinámica".

Algunos lenguajes proporcionan medios más estáticos (en “tiempo de compilación”) de polimorfismo.

- **Herencia:** la herencia es la propiedad por la cual una clase asume como propios todos los atributos y métodos definidos por otra. La que hereda se denomina «**subclase**» o «clase derivada». La que transmite la herencia recibe el nombre de «**superclase**» o «clase base».

La POO difiere de la programación estructurada tradicional, en la que los datos y los procedimientos están separados y sin relación, ya que lo único que se busca es el procesamiento de unos datos de entrada para obtener otros de salida. La programación estructurada anima al programador a pensar sobre todo en términos de procedimientos o funciones, y, en segundo lugar, en las estructuras de datos que esos procedimientos manejan. En la programación estructurada solo se escriben funciones que procesan datos. Los programadores que emplean POO, en cambio, primero definen objetos para luego enviarles mensajes solicitándoles que realicen sus métodos por sí mismos.

Lenguajes: Smalltalk, C++, C#, Java, Python, Eiffel, Ruby, Scala, Swift, VB.NET, PHP, D, R, Object Pascal, Gambas, JavaScript, Oz, Perl, Processing, ActionScript.

**Programación FUNCIONAL:** es un paradigma de **programación declarativa** basado en la utilización de funciones aritméticas que no maneja datos mutables o de estado. En la práctica, la diferencia entre una función matemática y la noción de una "función" utilizada en la programación imperativa es que las funciones imperativas pueden tener efectos secundarios, al cambiar el valor de cálculos realizados previamente. Por esta razón carecen de transparencia referencial, es decir, la misma expresión sintáctica puede resultar en valores diferentes en diferentes momentos dependiendo del estado del programa siendo ejecutado. Con código funcional, en contraste, el valor generado por una función depende exclusivamente de los argumentos alimentados a la función. Al eliminar los efectos secundarios se puede entender y predecir el comportamiento de un programa mucho más fácilmente, y esta es una de las principales motivaciones para utilizar la programación funcional.

Los programas escritos en un lenguaje funcional están constituidos únicamente por definiciones de funciones, entendiendo éstas no como subprogramas clásicos de un lenguaje imperativo, sino como funciones puramente matemáticas, en las que se verifican ciertas propiedades como la transparencia referencial (el significado de una expresión depende únicamente del significado de sus subexpresiones), y por tanto, la carencia total de efectos colaterales. Se denominan funciones puras.

Un tipo especial de funciones son las funciones de orden superior, que pueden tomar otras funciones como argumentos o devolverlos como resultados.

Otras características propias de estos lenguajes son la no existencia de asignaciones de variables y la falta de construcciones estructuradas como la secuencia o la iteración, lo que obliga en la práctica a que todas las repeticiones de instrucciones se lleven a cabo por medio de funciones recursivas.

En definitiva, un programa dentro del paradigma funcional es una función o un grupo de funciones compuestas por funciones más simples estableciéndose que una función puede llamar a otra, o el resultado de una función puede ser usado como argumento de otra función.

Podemos distinguir 2 tipos de programación funcional según el modelo de cómputo en el que están basados:

- Cálculo lambda: sus características son que están basado en s-expresiones, son de tipado débil y admite la metaprogramación. Lenguajes: familia Lisp, Scheme.
- Lógica combinatoria: mantienen un sistema estricto de tipos, implementan la transparencia referencial y la evaluación perezosa. Lenguajes: familia ML, Haskell.

#### Lenguajes:

- Puros: Haskell, Miranda.
- Híbridos: Lisp, Scheme, Scala, familia ML, F# (Microsoft).

#### Ejemplo: resolución de un sudoku en Haskell

```
fila tablero fil = [ w | (x,y,z,w) <- tablero, x==fil, w/=0]

columna tablero col = [ w | (x,y,z,w) <- tablero, y==col, w/=0]

region tablero reg = [ w | (x,y,z,w) <- tablero, z==reg, w/=0]

candidatos tablero (f,c,r,v) n = [ valor | valor <- [1..n], not(valor `elem`
(valores_fila)+(valores_columna)+(valores_region)) ]
  where valores_fila = fila tablero f
        valores_columna = columna tablero c
        valores_region = region tablero r

casillasVacias [] = []
casillasVacias ((f,c,r,v):xs)
  | v == 0 = (f,c,r,v) : casillasVacias xs
  | otherwise = casillasVacias xs

esSol (tablero,casillas,n) = casillas ==[]

modificaT (f,c,r,v) ((x,y,z,w):xs) valor
  | f==x && c==y = (x,y,z,valor) : xs
  | otherwise = (x,y,z,w) : modificaT (f,c,r,v) xs valor

hijos (tablero,((f,c,r,v):ys),n) = [ ((modificaT (f,c,r,v) tablero valor),(ys),n) |
valor <- candidatos tablero (f,c,r,v) n]

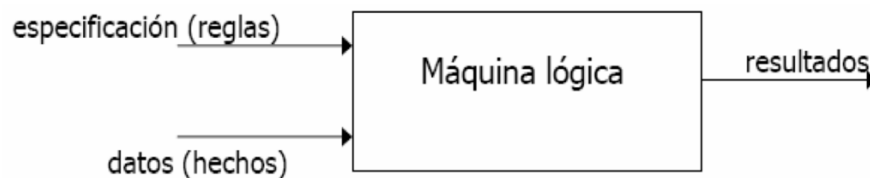
obtenerTableros [] = []
obtenerTableros ((tablero,casillas,n):xs) = [ (a,b,c,d) | (a,b,c,d) <- tablero ] :
obtenerTableros xs

sudoku tablero n = imprimeSoluciones (obtenerTableros( bt esSol
hijos(tablero,casillasVacias tablero,n))) n
```

- **Programación LÓGICA**: es un paradigma de **programación declarativa** que gira en torno al concepto de predicado, o relación entre elementos. Está basado en la lógica de predicados de 1º orden. Su característica fundamental es que permite al software

“razonar”, ya sean razonamientos deductivos (usa principios generales para llegar a una conclusión específica) o inductivos (utiliza premisas particulares para llegar a una conclusión general). También pueden incorporar algunos comportamientos de orden superior como la lógica difusa.

Dada una base de datos consistente en un conjunto de entidades, propiedades de esas entidades y relaciones de unas entidades con otras, el sistema es capaz de hacer razonamientos. Básicamente, este proceso se expresa de la siguiente forma:



$$\text{Resultados} = \text{reglas} + \text{hechos}$$

Donde entendemos que “hechos” es el conjunto de datos que conoce el sistema a priori (o que va adquiriendo a lo largo de su ejecución) y “reglas” son un conjunto de operaciones que se pueden aplicar a dichos datos para sacar un resultado lógico.

La programación lógica se dirige al desarrollo de aplicaciones en el campo de la IA: demostración automática de teoremas, sistemas expertos o lenguaje natural.

Lenguajes: Prolog.

#### Ejemplo: gestor de fichajes

##### **Reglas**

jugador(X,P,V): jugador con nombre X que juega en la posición P y que tiene un valor V.  
 crack(X): X es un crack.  
 internacional(X): X es un internacional. También, X es internacional si es un crack.  
 espaniol(X): X es español.  
 posicion(X,P): posición P del jugador con nombre X.  
 valor(X,V): valor V del jugador con nombre X.  
 valorInicial(X,V): valor inicial del jugador con nombre X.  
 amaColores(X): X ama los colores.  
 conflictivo(X): X es conflictivo.  
 precio(X,Valor): precio del jugador con nombre X.

##### **Hechos**

jugador(oblak,portero,30)  
 jugador(pedri,centrocampista,50)  
 jugador(benzema,delantero, 55)  
 crack(pedri)  
 internacional(benzema)

- **Programación DECLARATIVA:** se enfoca en describir lo que se quiere obtener en lugar de cómo hacerlo. Un ejemplo de lenguaje declarativo es SQL.

- **Programación ORIENTADA A ASPECTOS** (AOP, *Aspect Oriented Programming*): es un paradigma de programación que intenta formalizar y representar de forma concisa los elementos que son transversales a todo el sistema. En los lenguajes orientados a objetos, la estructura del sistema se basa en la idea de *clases y jerarquías de clases*. La herencia permite modularizar el sistema, eliminando la necesidad de duplicar código. No obstante, siempre hay aspectos que son transversales a esta estructura: el ejemplo más clásico es el de control de permisos de ejecución de ciertos métodos en una clase:

```
public class MiObjetoDeNegocio {  
    public void metodoDeNegocio1() throws SinPermisoException {  
        chequeaPermisos();  
        //resto del código  
        ...  
    }  
  
    public void metodoDeNegocio2() throws SinPermisoException {  
        chequeaPermisos();  
        //resto del código  
        ...  
    }  
  
    protected void chequeaPermisos() throws SinPermisoException {  
        //chequear permisos de ejecucion  
        ...  
    }  
}
```

Como vemos, estructurando adecuadamente el programa se puede *minimizar* la repetición de código, pero es prácticamente imposible eliminarla. La situación se agravaría si además tuviéramos que controlar permisos en objetos de varias clases. El problema es que en un lenguaje orientado a objetos los aspectos transversales a la jerarquía de clases no son modularizables ni se pueden formular de manera concisa con las construcciones del lenguaje. La programación orientada a aspectos intenta formular conceptos y diseñar construcciones del lenguaje que permitan modelar estos aspectos transversales sin duplicación de código. En nuestro ejemplo, se necesitaría poder especificar de alguna manera concisa que *antes* de ejecutar *ciertos métodos* hay que llamar a *cierto código*.

En AOP, a los elementos que son transversales a la estructura del sistema y se pueden modularizar gracias a las construcciones que aporta el paradigma se les denomina **aspectos** (*aspects*). En el ejemplo anterior el control de permisos de ejecución, modularizado mediante AOP, sería un aspecto.

Un **consejo** (*advice*) es una acción que hay que ejecutar en determinado/s punto/s de un código, para conseguir implementar un aspecto. En nuestro ejemplo, la acción a ejecutar sería la llamada a `chequeaPermisos()`. El conjunto de puntos del código donde se debe ejecutar un *advice* se conoce como **punto de corte** o *pointcut*. En nuestro caso serían los métodos `metodoDeNegocio1()` y `metodoDeNegocio2()`. Nótese que aunque se hable de "punto de corte" en singular, en general no es un único punto del código.

Lenguajes: AspectJ (Java), AspectC++ (C++), Aspect (Perl), phpAspect (PHP).

- **Programación DIRIGIDA POR EVENTOS:** la ejecución del programa no es determinista (programación imperativa), sino que sucede en función de los sucesos (eventos) que ocurren.

La estructura básica de un programa es un bucle sin fin, del cual solo se sale para terminar la aplicación. Lo que ocurre dentro de este bucle está determinado por los sucesos (eventos) que ocurren como consecuencia de la interacción con el mundo exterior, con el entorno en el que se ejecuta la aplicación. Un evento representa cualquier cambio significativo en el estado del programa. El programador decide cuáles son los tipos de eventos que requieren una respuesta por parte de la aplicación, definiendo un código concreto (manejador).

```
private void button1_Click(object sender, RoutedEventArgs e)
{
    // realizo un casting del elemento seleccionado de la grilla lo
    // combinto a una persona.
    Persona personaSeleccionada = (Persona) (dataGrid1.SelectedItem);

    if(personaSeleccionada!=null)
    {
        //Si la persona es distinto de null lanzo el evento, esto
        //lleva el nombre a la otra pantallita.
        OnMiEntidadSeleccionada(personaSeleccionada);
    }
    else
    {
        MessageBox.Show("Seleccione 1 persona", "Aviso");
    }
}
```

Este tipo de programación permite el uso de GUI, las cuales pueden ser diseñadas por multitud de lenguajes de programación.

Lenguajes: JavaScript, C#, VB.NET, Java, ASP.NET.

**Programación por RESTRICCIONES** (Constraints Programming): dedicado a problemas que impliquen una búsqueda compleja de soluciones. En lugar de implementar nosotros mismos los algoritmos, en programación con restricciones nos limitamos a especificar el problema, dejando a un programa externo, el resolutor, la tarea de encontrar la solución. El paradigma apareció en el contexto de la programación lógica.

La programación basada en restricciones se basa en crear un modelo formado por variables, para las cuales deberemos indicar qué posibles valores pueden tomar, y restricciones, que expresan relaciones que deben cumplirse entre esas variables. Una vez creado el modelo, el sistema se encarga de encontrar aquellas soluciones que se ajustan a las restricciones.

Por ejemplo, el problema de 8 reinas, que consiste en colocar sobre un tablero de ajedrez 8 reinas sin que ninguna amenace a las demás. Modelar ese problema a base de restricciones es posible:

- 1)  $(x_i, y_i)$  es la posición de la reina  $i$ -ésima.
- 2)  $x_i$  es un valor entre 0 y 7
- 3)  $y_i$  es un valor entre 0 y 7
- 4) Todos los  $x_i$  son distintos (i.e. no hay dos reinas en la misma fila)
- 5) Todos los  $y_i$  son distintos (i.e. no hay dos reinas en la misma columna)
- 6) Todos los  $x_i - y_i$  son distintos (i.e. no hay dos reinas en la misma diagonal de arriba-izquierda a abajo-derecha)
- 7) Todos los  $x_i + y_i$  son distintos (i.e. no hay dos reinas en la misma diagonal de arriba-derecha a abajo-izquierda)

Lenguajes: basados en Prolog, Mozart.

- **Programación REACTIVA:** es un enfoque de programación asíncrona que se centra en la propagación de cambios y el manejo de flujos de datos. Este paradigma permite a los sistemas reaccionar a eventos o cambios en el estado de manera eficiente y escalable. Esto es porque siguen el patrón Observer, del tal forma que cuando hay un cambio de estado en un objeto, los otros objetos son notificados y actualizados. Por lo tanto, en lugar de estar preguntando los cambios, los eventos se realizan de forma asíncrona para que los observadores puedan procesarlos.

**ReactiveX** es un API que extiende el patrón observador que facilita el manejo de flujo de datos y eventos a través de la combinación del patrón Observer e Iterator. RxJava para Java, RxPy para Python, Rx.NET para C# y RxPHP para PHP.

- **Programación REFLEXIVA:** es un enfoque en el que un programa tiene la capacidad de examinar y modificar su propia estructura y comportamiento en tiempo de ejecución. Esta capacidad permite que el programa se adapte dinámicamente a diferentes situaciones y requisitos.

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        print(f'Hola, {self.nombre}')
```

```
persona = Persona('Maria')
persona.saludar()

# Reflexión
atributo = getattr(persona, 'nombre')
print(f'Nombre obtenido mediante reflexión: {atributo}')
```

- **Programación CONCURRENTE:** se enfoca en la ejecución simultánea de procesos o hilos.

Lenguajes: Erlang, Go.

#### (4) Según su TRADUCCIÓN

Según la manera y el momento de obtener el código objeto a partir del código fuente, encontramos dos tipos de lenguajes:

- **COMPILADOS:** C.
- **INTERPRETADOS:** JavaScript.
- **MIXTOS:** como es el caso de Java. Primero pasan por una fase de compilación en la que el código fuente se transforma en “bytecode”, y este puede ser posteriormente ejecutado (interpretado) en distintas arquitecturas gracias a una máquina virtual.

### (5) Según la clase de TIPADO

Según el grado de conversión/sustitución entre tipos que nos permita un lenguaje, nos encontraremos los siguientes tipos de lenguajes:

- **Lenguajes FUERTEMENTE tipados:** no permiten apenas conversión implícita entre tipos, solo algunos casos considerados como "seguros" y mediante una conversión explícita, (casting en Java), por ejemplo, de un entero a un decimal, donde no se perdería información.

Lenguajes: C, C++, Java, Python, Haskell, ML.

- **Lenguajes DÉBILMENTE tipados:** son más permisivos en la conversión de tipos, permitiendo conversiones implícitas entre ellos (por ejemplo, de string a float). La conversión se realiza de manera implícita o explícita y dependerá del lenguaje y sus reglas.

Lenguajes: PHP, Javascript, Lisp, Prolog, Perl.

Comúnmente, se entiende que un lenguaje tiene un sistema de tipos más fuerte si para una determinada operación, que requiera un determinado tipo de datos, el lenguaje no acepta tipos que pudieran ser "compatibles". Existen distintos grados entre un lenguaje fuertemente tipado y un lenguaje débilmente tipado. Es por ello que la línea que divide a ambos es controvertida y a veces difícil de definir.

En un extremo tendríamos, por ejemplo, un lenguaje que nos permite declarar un método o función que acepte como parámetro un número real de doble precisión (es decir, un *double*), consideraríamos que el lenguaje **está muy fuertemente tipado si sólo** permitiese llamadas o invocaciones a esa función o método pasándole como parámetro un dato con un tipo *double*.

A partir de ahí, podríamos ir rebajando el listón. Prácticamente todos los lenguajes permiten una cierta conversión implícita de tipos, especialmente si se pueden considerar **seguras**. Por ejemplo, convertir un entero a un double suele ser una operación segura, dado que un entero no tiene decimales, y un double sí, es decir, es un tipo más amplio. Una conversión de este tipo puede ser implícita con un alto grado de seguridad. Al revés no es así. Si tenemos el tipo de datos double (por ejemplo 2,68) y se utiliza en un contexto en el que se requiere un entero, ¿qué debería hacer el lenguaje con los decimales? ¿Los elimina? ¿Los redondea?

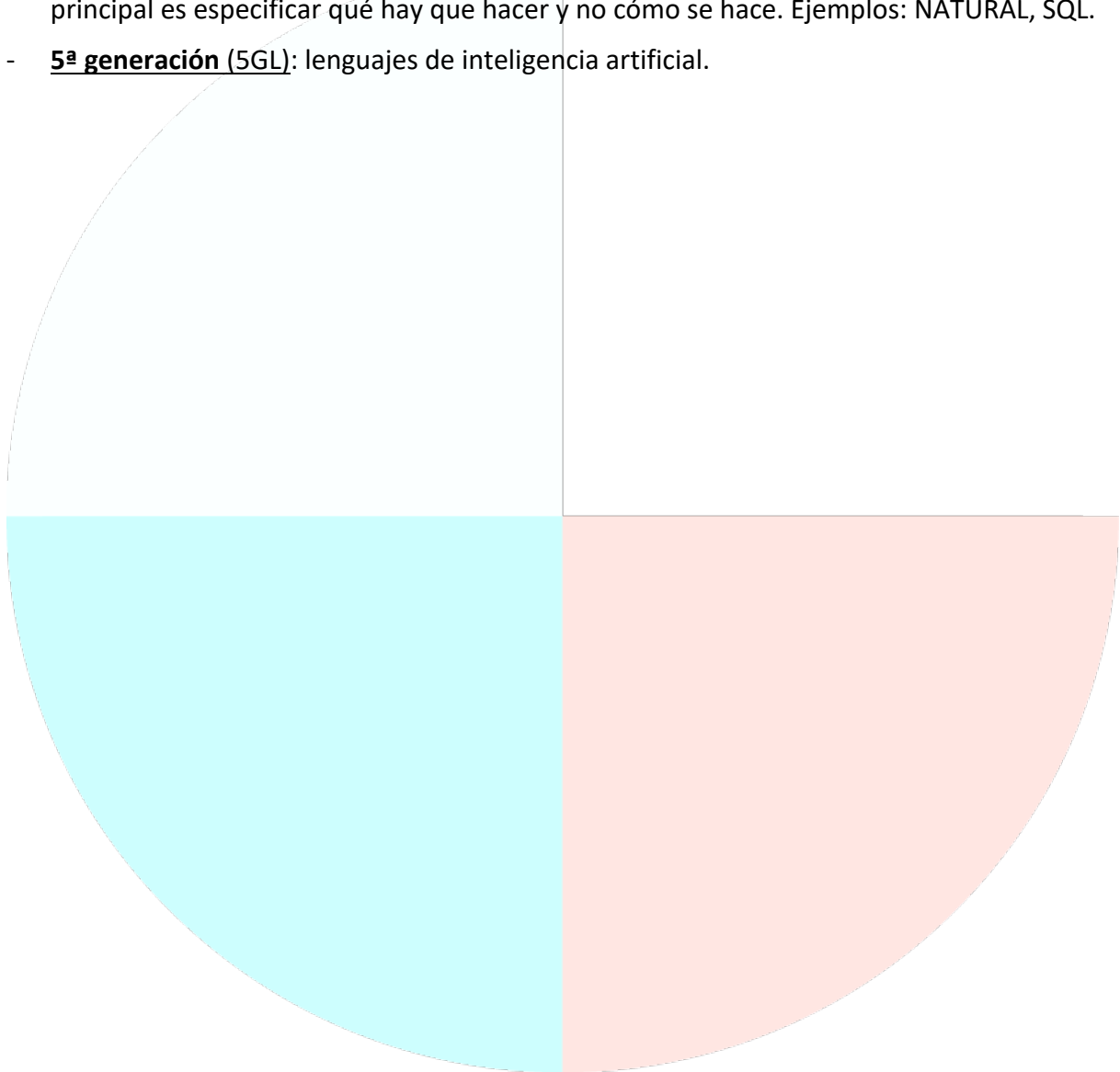
Un lenguaje **débilmente tipado**, en general, permite este tipo de conversiones implícitas inseguras. Es su filosofía, que el programador se encargue de programar, y el lenguaje se encargará de hacer todo lo posible para que el programa ejecute, liberando al programador de llevar también la cuenta de posibles conversiones de tipos.

Esta forma de actuar proporciona, sin duda, una manera muy sencilla de programar. Es decir, en un lenguaje **débilmente tipado**, el lenguaje prestará poca atención a una definición estricta de los tipos. Eso hace que el programador pueda centrarse en el objetivo del problema que resuelve, que el lenguaje intentará encargarse de todo lo referido a los tipos, sin embargo, la consecuencia lógica es que los posibles errores se detectarán más tarde, en tiempo de ejecución.

**(6) Según la GENERACIÓN**

Desde el punto de vista histórico podemos clasificar los lenguajes de programación según la generación:

- **1ª generación (1GL)**: lenguaje máquina.
- **2ª generación (2GL)**: lenguajes simbólicos.
- **3ª generación (3GL)**: lenguajes de medio y alto nivel.
- **4ª generación (4GL)**: lenguajes utilizados para propósitos específicos. La característica principal es especificar qué hay que hacer y no cómo se hace. Ejemplos: NATURAL, SQL.
- **5ª generación (5GL)**: lenguajes de inteligencia artificial.



## 5. Principales lenguajes de programación

| Lenguaje   | Tipo         | Paradigma                                                 | Uso                                                                                                |
|------------|--------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| ABAP       | Interpretado | Multiparadigma                                            | Gestión empresarial                                                                                |
| AspectJ    | Compilado    | Orientado a aspectos                                      | General                                                                                            |
| ASP.NET    | Interpretado | Multiparadigma                                            | Web                                                                                                |
| C          | Compilado    | Imperativo                                                | General                                                                                            |
| C++        | Compilado    | Multiparadigma<br>Imperativo y OO                         | General                                                                                            |
| C#         | Compilado    | Multiparadigma<br>Imperativo, OO, funcional y reflexivo   | General                                                                                            |
| Clojure    | Compilado    | Funcional                                                 | General                                                                                            |
| D          | Compilado    | Multiparadigma (OO)<br>Imperativo, OO y funcional         | General                                                                                            |
| Dart       | Compilado    | Multiparadigma                                            | Web. Desarrollado por Google                                                                       |
| Eiffel     | Compilado    | OO                                                        | General                                                                                            |
| Erlang     | Interpretado | Multiparadigma<br>OO, funcional y concurrente             | Programación concurrente                                                                           |
| Etoys      |              | OO                                                        | Educación                                                                                          |
| F#         | Compilador   | Multiparadigma<br>Imperativo, OO y funcional              | General (.NET). Orientado a prog. funcional                                                        |
| Gambas     | Compilado    | Multiparadigma                                            | General                                                                                            |
| GO         | Compilado    | Multiparadigma<br>Imperativo, OO y concurrente            | Web (Google)                                                                                       |
| Groovy     | Compilado    | Multiparadigma<br>Imperativo, OO y funcional              | General                                                                                            |
| Haskell    | Compilado    | Funcional                                                 | IA                                                                                                 |
| Java       | Compilado    | OO                                                        | General                                                                                            |
| JavaScript | Interpretado | Multiparadigma<br>Imperativo, OO (prototipos) y funcional | Web                                                                                                |
| J#         |              | OO                                                        | Web                                                                                                |
| Kotlin     | Compilado    | OO                                                        | Aplicaciones para Android.<br>Se compila a código intermedio para JVM o para intérprete JavaScript |
| Lisp       | Compilado    | Funcional                                                 | IA                                                                                                 |
| Lua        | Interpretado | Imperativo                                                | Educación                                                                                          |
| Miranda    | Interpretado | Funcional                                                 | IA                                                                                                 |

|                    |              |                                                                               |                                              |
|--------------------|--------------|-------------------------------------------------------------------------------|----------------------------------------------|
| <b>ML</b>          | Compilado    | Funcional                                                                     | IA                                           |
| <b>Objective-C</b> | Compilado    | OO                                                                            | General                                      |
| <b>Oz</b>          | Compilado    | Multiparadigma                                                                | Programación distribuida y por restricciones |
| <b>P#</b>          | Compilado    | Lógica                                                                        | General (.NET)                               |
| <b>Perl</b>        | Interpretado | <i>Multiparadigma</i><br>Imperativo, OO y funcional                           | General                                      |
| <b>PHP</b>         | Interpretado | <i>Multiparadigma</i><br>Imperativo, OO, funcional y reflexivo                | Web                                          |
| <b>Processing</b>  | Compilado    | OO                                                                            | Educación                                    |
| <b>Prolog</b>      | Interpretado | Lógico                                                                        | IA                                           |
| <b>Python</b>      | Interpretado | <i>Multiparadigma</i><br>Imperativo, OO, funcional y reflexivo                | General                                      |
| <b>R</b>           | Interpretado | <i>Multiparadigma</i><br>Imperativo, OO y funcional                           | Estadísticas                                 |
| <b>Racket</b>      | Compilado    | <i>Multiparadigma</i><br>Imperativo, OO, funcional y lógico                   | Educación                                    |
| <b>Ruby</b>        | Interpretado | <i>Multiparadigma (OO)</i><br>Imperativo, OO, funcional y reflexivo           | General                                      |
| <b>Scala</b>       | Compilado    | <i>Multiparadigma</i><br>Imperativo, OO y funcional                           | General (WS)                                 |
| <b>Scheme</b>      | Compilado    | Funcional                                                                     | IA                                           |
| <b>Scratch</b>     | Compilado    | Imperativo                                                                    | Educación. Programación visual               |
| <b>Smalltalk</b>   | Compilado    | OO                                                                            | General                                      |
| <b>Swift</b>       | Compilado    | <i>Multiparadigma</i><br>Imperativo, OO, funcional, declarativo y concurrente | General (Apple)                              |
| <b>TypeScript</b>  | Compilado    | <i>Multiparadigma</i><br>Imperativo, OO, funcional y declarativo              | Web (Microsoft)<br>Compila a JavaScript      |
| <b>Vala</b>        | Compilado    | <i>Multiparadigma</i><br>Imperativo, OO y funcional                           | General                                      |
| <b>VB.NET</b>      | Compilado    | OO                                                                            | Web                                          |

## 2. REPRESENTACIÓN DE TIPOS DE DATOS

Los **tipos de datos** definen el modo en que se usa el espacio (memoria) en los programas. Especificando un tipo de datos, se está indicando al compilador cómo crear un espacio de almacenamiento en particular y también cómo manipular este espacio.

Los tipos de datos pueden estar predefinidos o abstractos. Un tipo de dato predefinido es intrínsecamente comprendido por el compilador. En contraposición, un tipo de datos definido por el usuario es aquel que se crea como una clase. Estos se denominan comúnmente tipos de datos abstractos.

El compilador sabe cómo manejar tipos predefinidos por sí mismo y maneja los tipos de datos abstractos leyendo la definición de la clase.

### Tipos de datos primitivos (Java)

| TIPO           | DESCRIPCIÓN              | DEFAULT | TAMAÑO  | RANGO                  |
|----------------|--------------------------|---------|---------|------------------------|
| <b>boolean</b> | true o false             | false   | 8 bits  | true, false            |
| <b>byte</b>    | entero complemento a dos | 0       | 8 bits  | -128 a 127             |
| <b>char</b>    | carácter Unicode         | \u0000  | 16 bits |                        |
| <b>short</b>   | entero complemento a dos | 0       | 16 bits | -32768 a 32767         |
| <b>int</b>     | entero complemento a dos | 0       | 32 bits | $-2^{31}$ a $2^{31}-1$ |
| <b>long</b>    | entero complemento a dos | 0       | 64 bits | $-2^{63}$ a $2^{63}-1$ |
| <b>float</b>   | coma flotante IEEE 754   | 0.0     | 32 bits |                        |
| <b>double</b>  | coma flotante IEEE 754   | 0.0     | 64 bits |                        |

Los tipos de datos primitivos se pueden organizar en 4 grupos:

- Numéricos enteros: son los tipos byte, short, int y long. Los 4 representan números enteros con signo.
- Carácter: el tipo char representa un carácter codificado en el sistema Unicode.
- Numérico decimal: los tipos float y double representan números decimales en coma flotante.
- Lógicos: el tipo boolean es el tipo de dato lógico; los dos únicos posibles valores que puede representar un dato lógico son true y false. *true* y *false* son palabras reservadas.

### Otros tipos de datos

Además de los tipos predefinidos, el programador puede crear tipos de datos a partir de los tipos ya existentes, es decir, estructuras del tipo struct o dependiendo del lenguaje definir

clases que posteriormente permite la instanciación de objetos como nuevos tipos de datos. Muchos lenguajes de programación disponen de tipos complejos o clases, estructuradas en una jerarquía, que amplían el repertorio de tipos de datos disponibles.

### Conversión de tipos de datos

La conversión entre tipos puede realizarse de manera implícita o explícita, según el caso.

Para ello tendremos que acudir al concepto de "casting" (convertir): es el proceso de transformar un valor de un tipo a un valor de otro tipo. Dependiendo del lenguaje, como hemos visto en el caso anterior, existirá esta diferenciación entre conversiones implícitas y explícitas, y serán de una forma u otra.

Normalmente:

- Los lenguajes fuertemente tipados tendrán conversiones implícitas y explícitas.
- Los lenguajes débilmente tipados solamente las conversiones implícitas.

Conversión Implícita: por lo general no es necesario utilizar ninguna notación de conversión explícita al asignar un tipo de dato primitivo más pequeño a un tipo más grande. Por ejemplo, se puede asignar un valor byte (que puede tener valores entre -128 a 127) a una variable tipo short (que tiene valores entre -32768 a 32767), sin necesidad de una conversión explícita.

A continuación, se presenta un resumen de algunas de las asignaciones que no requieren de una conversión explícita (por ejemplo, en el caso de Java). Un tipo puede ser asignado a cualquiera de los tipos que están a su izquierda:

double ← float  
long ← int ← short ← byte

Conversión Explícita: mediante la conversión explícita se le indica al compilador que es intencional la conversión (y que acepta las consecuencias de cualquier pérdida de precisión).

El formato general de conversión es: (tipo) valor\_a\_convertir

Para hacer una conversión explícita, sólo debe rodear al tipo de variable con paréntesis, por ejemplo (float).

### Tipado estático o dinámico

En un lenguaje **ESTÁTICAMENTE tipado**, cada variable debe ser declarada con un tipo. Eso ocurre por ejemplo en C# o Java. Para utilizar una variable de tipo entero, es necesario indicar que en efecto, es de ese tipo. **El tipado estático es típico de los lenguajes compilados.**

El tipo no se altera hasta que la variable se extingue. Este comportamiento se aplica tanto a variables locales (a un método o función), a parámetros (de un método o función), a variables

de instancia (de un objeto), a variables de clase ("static") o a variables globales (en lenguajes no orientados a objetos).

Por ejemplo, en C# o Java, para utilizar una variable de nombre "i" que contenga un entero:

```
int i; //Primero se declara
i=3; //y luego se utiliza, aunque se podría haber hecho en la misma línea
```

No es posible asignar un valor de otro tipo a una variable de un tipo concreto:

```
i="hola"; //incorrecto, ya que no se ha especificado el tipo de dato
```

En un lenguaje **DINÁMICAMENTE tipado**, no suele ser necesario declarar el tipo de las variables. Es típico de los lenguajes interpretados. Ocurre en lenguajes como PHP o Python. Las variables empiezan a existir cuando se les da valor. Los lenguajes dinámicamente tipados tienen en cuenta los tipos, pero no de las variables, sino de su contenido.

Una misma variable puede contener en un instante dado un dato de un determinado tipo, y en otro instante, puede contener otro dato de otro tipo.

Por ejemplo, en Python, podemos utilizar una variable "i" para contener un entero, sin necesidad de indicar que va a contener un entero (la almohadilla # indica un comentario):

```
i=3 #la variable comienza a existir al darle valor
i="hola" #luego, puede contener un valor de otro tipo
```

Con el tipado estático, dadas dos variables enteras, si después queremos hacer una suma, *seguro* que se puede hacer sin tener que invertir tiempo en comprobarlo, dado que el compilador sabe que *seguro* que contienen un entero, por lo tanto, la suma se puede hacer sin comprobaciones durante la ejecución.

```
int i1=3;
int i2=5;
int i3=i1+i2; //No se hacen comprobaciones en la suma:
               //seguro que i1 e i2 contienen un entero.
```

Con el tipado dinámico, las operaciones son algo más costosas, el intérprete deberá obtener el contenido de las variables y decidir qué operación hacer y cómo, suponiendo que ésta sea posible. El qué hacer dependerá del lenguaje.

En Python, la 3ª línea necesita comprobaciones adicionales a nivel interno, con respecto al tipado estático:

```
i1=3
i2=5
i3=i1+i2
```

## 3. OPERADORES

Los operadores son símbolos (+, \*...) que permiten hacer operaciones sobre datos, generando un resultado. Pueden ser unarios (aplican a un operando), binarios e incluso ternarios.

### Operadores aritméticos

| OPERADOR          | SIGNIFICADO                    |
|-------------------|--------------------------------|
| +                 | Suma                           |
| -                 | Resta                          |
| *                 | Multiplicación                 |
| /                 | División                       |
| %                 | Módulo (resto división entera) |
| <b>++variable</b> | Pre-incremento                 |
| <b>variable++</b> | Post-incremento                |
| <b>--variable</b> | Pre-decremento                 |
| <b>variable--</b> | Post-decremento                |

Ejemplo: Indique la salida de las siguientes instrucciones.

```
int a = 20, b = 10, c = 0, d = 20, e = 40, f = 30;  
String x = "Thank", y = "You";
```

```
System.out.println("a + b = " + (a + b));  
System.out.println("a - b = " + (a - b));
```

```
System.out.println("x + y = " + x + y);
```

```
System.out.println("a * b = " + (a * b));  
System.out.println("a / b = " + (a / b));  
System.out.println("a % b = " + (a % b));
```

```
c = ++a;  
System.out.println("Valor de c (++a) = " + c);
```

```
c = b++;  
System.out.println("Valor de c (b++) = " + c);  
c = --e;  
System.out.println("Valor c (--e) = " + c);
```

**Operadores relacionales**

| OPERADOR | SIGNIFICADO       |
|----------|-------------------|
| <        | Menor que         |
| <=       | Menor o igual que |
| ==       | Igual a           |
| >        | Mayor que         |
| >=       | Mayor o igual que |
| !=       | Distinto          |

**Operadores lógicos**

| OPERADOR | SIGNIFICADO |
|----------|-------------|
| &&       | AND         |
|          | OR          |
| !        | NOT         |

**Ejemplo:** Indique la salida de las siguientes instrucciones.

```
boolean condition = true;  
System.out.println("Valor de !condition = " + !condition);
```

**Operador ternario**

| OPERADOR                    | SIGNIFICADO                                                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| condición?si true: si false | Si la condición se evalúa como verdadera se ejecuta la instrucción después del ?, en caso contrario, la instrucción después de : |

**Ejemplo:** Indique el valor de la variable result.

```
int a = 20, b = 10, c = 30, result;  
result = ((a > b) ? (a > c) ? a : c : (b > c) ? b : c);
```

**Operadores a nivel de bits**

| OPERADOR | SIGNIFICADO                           |
|----------|---------------------------------------|
|          | OR                                    |
| &        | AND                                   |
| >>       | Desplazamiento de bits a la derecha   |
| <<       | Desplazamiento de bits a la izquierda |
| ^        | XOR de bits                           |
| ~        | NOT de bits                           |

**Ejemplo:** Indique la salida de las siguientes instrucciones.

```
int a = 0x0005; // 0101
int b = 0x0007; // 0111
System.out.println("a&b = " + (a & b));
System.out.println("a|b = " + (a | b));
System.out.println("a^b = " + (a ^ b));
System.out.println("~a = " + ~a);
System.out.println("a<<2 = " + (a << 2));
System.out.println("a>>2 = " + (a >> 2));
```

**Operador de asignación**

| OPERADOR | SIGNIFICADO                             |
|----------|-----------------------------------------|
| =        | Asignación                              |
| +=       | Incremento y asignación                 |
| -=       | Decremento y asignación                 |
| %=       | Módulo y asignación                     |
| *=       | Multiplicación y asignación             |
| /=       | División y asignación                   |
| ^=       | XOR a nivel de bits y asignación        |
| >>=      | Desplazar bits a derecha y asignación   |
| <<=      | Desplazar bits a izquierda y asignación |
| &=       | AND a nivel de bits y asignación        |
| =        | OR a nivel de bits y asignación         |

Ejemplo: Indique la salida de las siguientes instrucciones.

```
int a = 20, b = 10, c, d, e = 10, f = 4;
a += 1;
b -= 1;
e *= 2;
f /= 2;
System.out.println("a,b,e,f (usando operadores cortos)= "+a+", "+b+", "+e+", "+f);
```

```
int a = 0x0005; // 0101
int b = 0x0007; // 0111
a &= b;
System.out.println("a= " + a);
```

### Operador escape

Permite incluir en la salida diferentes secuencias de escape:

| OPERADOR | SIGNIFICADO     |
|----------|-----------------|
| \n       | Nueva línea     |
| \t       | Tabulador       |
| \\       | Barra invertida |
| \'       | Comilla simple  |
| \"       | Comilla doble   |

Ejemplo: Indique la salida del siguiente código.

```
String cabecera = "\n\tPRONOSTICO DE CLIMA:\n" ;
cabecera += "\n\tDia\t\tMañana\tNoche\tCondiciones\n" ;
cabecera += "\t---\t\t-----\t----\t-----\n" ;
String pronostico = "\tDomingo\t25C\t\t23C\t\tSoleado\n";
pronostico += "\tLunes\t24C\t\t19C\t\tSoleado\n";
pronostico += "\tMartes\t26C\t\t15C\t\tNublado\n";
System.out.print(cabecera+pronostico);
```

Una vez vistos los operadores que generalmente son utilizados en los lenguajes de programación, es conveniente conocer la **precedencia**, es decir, dada una expresión cualquiera que incluya varios operadores, cuál es el orden de aplicación de los mismos (su prioridad). De esta forma, establecemos el siguiente orden general:

- |                 |                           |
|-----------------|---------------------------|
| 1. (), [], .    | 6. ==, !=                 |
| 2. --, ++, !    | 7. &&                     |
| 3. *, /, %      | 8.                        |
| 4. +, -         | 9. ?:                     |
| 5. >, >=, <, <= | 10. =, +=, -=, *=, /=, %= |

## 4. INSTRUCCIONES CONDICIONALES

### if

La sentencia `if` es la declaración de toma de decisiones más simple. Se usa para decidir si una determinada declaración o bloque de enunciados se ejecutará o no; es decir, si una determinada condición es verdadera (`true`), se ejecutará un bloque de enunciado y, de ser falsa (`false`), no.

La sintaxis es:

```
if (condición)
{
    // Declaraciones para ejecutar si
    // la condición es verdadera
}
```

Aquí, la condición después de la evaluación será verdadera o falsa. `if` acepta valores booleanos: si el valor es verdadero, ejecutará el bloque de instrucciones debajo de él.

**IMPORTANTE:** Si no se especifican las llaves '`{`' y '`}`' después del `if` (condición), entonces de forma predeterminada la sentencia `if` considerará que la declaración inmediatamente siguiente está dentro de su bloque.

Ejemplo:

```
if (condicion)
    declaracion1;
    declaracion2;

// Aquí si la condición es verdadera, el bloque if solo considerará que la
// declaracion1 está dentro de su bloque.
```

### if-else

La declaración `if` solo nos dice que, si una condición es verdadera ejecutará un bloque de instrucciones y si la condición es falsa, no lo hará. Pero, ¿y si queremos hacer otra cosa cuando la condición sea falsa? Aquí viene la declaración `else`. Podemos usar la instrucción `else` con la instrucción `if` para ejecutar un bloque de código cuando la condición es falsa.

La sintaxis es:

```
if (condición)
{
    // Ejecuta este bloque si
    // la condición es verdadera
}
else
{
    // Ejecuta este bloque si
    // la condición es falsa
}
```

**if anidado (nested-if)**

Un **if anidado** (nested-if) es una declaración **if** que se deriva de otro *if* o *else*. Las declaraciones **if** anidadas significan una instrucción *if* dentro de una declaración *if*.

La sintaxis es:

```
if (condicion1) {  
    // Ejecuta cuando condicion1 es verdadero  
    if (condicion2) {  
        // Ejecuta cuando condicion2 es verdadero  
    }  
}
```

```
if (condición)  
    declaración;  
else if (condición)  
    declaración;  
else  
    declaración;
```

**switch-case**

La instrucción **switch** es una declaración de bifurcación de múltiples vías (selección múltiple). Proporciona una forma sencilla de enviar la ejecución a diferentes partes del código en función del valor de la expresión.

La sintaxis es:

```
switch (expresión_a_evaluar) {  
    case valor1:  
        declaracion1;  
        break;  
    case value2:  
        declaracion2;  
        break;  
    ...  
    case valorN:  
        declaracionN;  
        break;  
    default:  
        declaracionDefault;  
}
```

**Ejemplo:**

```
int i = 9;  
switch (i) {  
    case 0: System.out.println("i es cero.");  
        break;  
    case 1: System.out.println("i es uno.");  
        break;  
    case 2: System.out.println("i es dos.");  
        break;  
    default: System.out.println("i es mayor que 2.");  
}
```

## 5. BUCLES

Un bucle en lenguajes de programación es una característica que facilita la ejecución de un conjunto de instrucciones repetidamente, mientras que algunas condiciones se evalúan como verdaderas.

### Bucle while

Un bucle **while** es una sentencia de control de flujo que permite que el código (cuerpo del bucle) se ejecute repetidamente en función de una condición booleana dada. El bucle **while** se puede considerar como una instrucción **if** repetitiva.

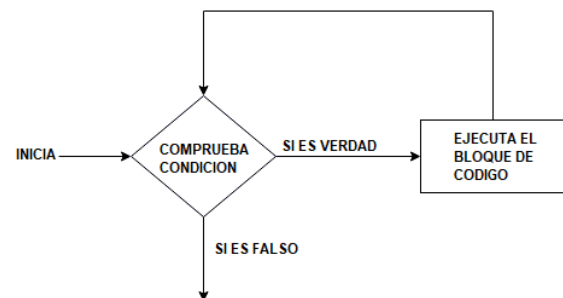
La sintaxis es:

```
while (condición) {
    cuerpo del bucle ...
}
```

Pasos:

1. **Condición de prueba:** se usa para probar la condición de salida de un bucle. Debe devolver un valor booleano. También es un bucle de control de entrada cuando se verifica la condición antes de la ejecución de las instrucciones de bucle.
2. **Si condición es false** Ir al **paso 5**, en otro caso ir al paso siguiente.
3. **Ejecución de instrucciones:** una vez que la condición se evalúa como verdadera, se ejecutan las instrucciones del cuerpo del bucle.
4. **Volver al paso 1.**
5. **Terminación de bucle:** cuando la condición se vuelve falsa, el bucle termina marcando el final de su ciclo de vida.

Pregunta: ¿Cuántas veces se ejecuta como mínimo un bucle while?

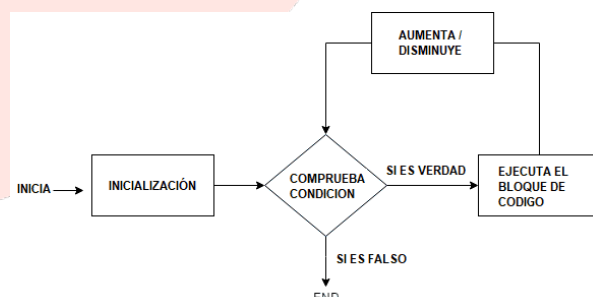


### Bucle for

El bucle **for** proporciona una forma concisa de escribir la estructura de bucle. A diferencia de un ciclo **while**, una sentencia **for** se permite la inicialización, la condición y el incremento/decremento en una línea, proporcionando así una estructura de bucle más corta y fácil de depurar.

La sintaxis es:

```
for (inicialización; condición prueba; incremento/decremento) {
    cuerpo
}
```



Pasos:

1. **Condición de inicialización:** Aquí, inicializamos la variable en uso. Marca el inicio de un ciclo **for**. Se puede usar una variable ya declarada o se puede declarar una variable, solo local para el bucle.
2. **Condición de prueba:** se usa para probar la condición de salida de un bucle. Debe devolver un valor booleano. También es un bucle de control de entrada cuando se verifica la condición antes de la ejecución de las instrucciones de bucle.
3. **Si condición es false Ir al paso 7**, en otro caso ir al paso siguiente.
4. **Ejecución de instrucciones:** una vez que la condición se evalúa como verdadera, se ejecutan las instrucciones del cuerpo del bucle.
5. **Incremento/Decremento:** se usa para actualizar la variable para la siguiente iteración.
6. **Volver al paso 2.**
7. **Terminación de bucle:** cuando la condición se vuelve falsa, el bucle termina marcando el final de su ciclo de vida.

Ejemplo: Indique el número de veces de ejecución de los siguientes bucles.

```
for (int x = 2; x <= 4; x++)
    System.out.println("Valor de x: " + x);

for (int x = 20; x <= 30; ++x)
    System.out.println("Valor de x: " + x);

for (int x = 10; x > 1; --x)
    System.out.println("Valor de x: " + x);

for (int i = 5; i != 0; i -= 2)
    System.out.println(i);
```

### Bucle for-each

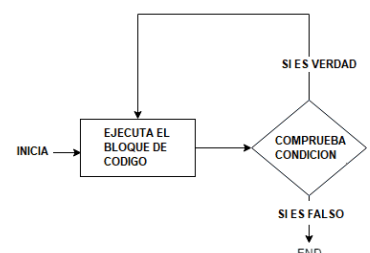
Este tipo de bucle **for** proporciona una forma más sencilla de iterar a través de los elementos de una colección o matriz. Es inflexible y debe usarse solo cuando existe la necesidad de recorrer los elementos de forma secuencial sin conocer el índice del elemento procesado actualmente.

La sintaxis es:

```
for (Elemento T:Colección de obj/array) {
    declaraciones
}
```

### Bucle do-while

El bucle **do while** es similar al **while** con la única diferencia de que comprueba la condición después de ejecutar las instrucciones, y por lo tanto es un ejemplo de *Exit Control Loop* (Salir del bloque de control).



La sintaxis es:

```
do {  
    //Cuerpo del bucle  
}  
while (condicion);
```

Pasos:

1. El bucle **do while** comienza con la ejecución del cuerpo. No hay verificación de ninguna condición la primera vez.
2. Verificación la condición.
3. Si condición es false ir al paso siguiente, en otro caso ir al paso 1.
4. Terminación del bucle.

Importante: tener en cuenta que el **bucle do-while** ejecutará el cuerpo **al menos 1 vez** antes de que se verifique cualquier condición y, por lo tanto, es un ejemplo de **bucle de control de salida**.

El concepto de recursividad ha sido ya estudiado en un tema anterior. Recordamos, que la recursividad consiste en definir problema en base a la propia definición.

Para la definición de funciones recursivas partimos de un problema complejo que dividimos en subproblemas de la misma naturaleza, del mismo tipo y de tamaño menor, los cuales son más sencillos de tratar. Se aplica la técnica divide y vencerás. Además, se exige especificar un caso base o de parada, que obtiene una solución inmediata, de tal manera que componemos los resultados a los subproblemas para obtener la solución al problema original.

Ejemplos:

```
int facR (int n) {  
    int resultado;  
    if (n==0) return 1;  
    resultado=facR(n-1)*n;  
    return resultado; }
```

```
int potencia (int x, int n) {  
    if (n==0) return 1;  
    else  
        return x * potencia(x,n-1); }
```

```
int cuentaNodos(NodoArbolBinario nodo) {  
    int contador=1;  
    if(nodo.getLeft()!=null) contador += cuentaNodos(nodo.getLeft());  
    if(nodo.getRight()!=null) contador += cuentaNodos(nodo.getRight());  
    return contador; }
```

```
int facI (int n) {  
    int t, resultado;  
    resultado=1;  
    for (t=1; t<=n; t++) resultado *=t;  
    return resultado; }
```

Recordamos, como inconvenientes de la recursividad, que las versiones recursivas de muchas rutinas pueden ejecutarse más lentamente que sus equivalentes iterativos debido a la sobrecarga adicional de las llamadas a métodos adicionales. Demasiadas llamadas recursivas a un método podrían causar un **desbordamiento de la pila (stack overflow)**.

## 6. PROCEDIMIENTOS, FUNCIONES Y PARÁMETROS

Los **procedimientos y funciones** son bloques de código reusables, que pueden ser invocados en cualquier parte de un programa, mediante expresiones que les representan. Son algoritmos específicos en un lenguaje de programación que resuelven un problema definido. Pueden ser usados cada vez que es requerido, simplemente colocando la expresión que les invoca, sin necesidad de reescribir todas las instrucciones que representan a tal procedimiento o función.

Por ejemplo, si se desea la raíz cuadrada de un número, podría usarse la expresión `Sqrt(x)`. La `x` dentro del paréntesis es llamada **argumento o parámetro**, siendo el número del cual se desea la raíz cuadrada. El argumento permite especificar la entrada a la función cuando el programa se está ejecutando. Siendo `x` la variable que almacena el número 12 para un programa, la función buscará la raíz cuadrada de 12. Cuando la función encuentra la raíz cuadrada, devuelve el valor al programa que la invocó. En algunos lenguajes los parámetros además de entrada pueden ser de salida o entrada/salida.

A los procedimientos y funciones en conjunto suele denominárseles también **rutinas o subrutinas**. Dependiendo del lenguaje de programación que se esté utilizando y de acuerdo con cómo éste implemente las rutinas, se pueden imponer algunas diferencias entre las funciones y los procedimientos. Las funciones, en general, son rutinas que devuelven un valor como resultado a partir de uno o más argumentos. Es el caso de las funciones matemáticas, que calculan el logaritmo de un número, su raíz cuadrada... Los procedimientos, por otro lado, son aquellas rutinas que no devuelven un valor, sino que realizan una acción determinada, usando tal vez un grupo de argumentos. Es el caso, por ejemplo, de un procedimiento para presentar por pantalla una circunferencia, en la que tendría que suministrarse simplemente valores como la posición en la pantalla y el radio de la misma. Aunque como hemos indicado, la rutina podría devolver un valor a través de un parámetro de entrada/salida o salida.

Por todo ello, el nombre de una función puede aparecer como parte de una expresión mientras que el nombre de un procedimiento debe aparecer como una instrucción independiente, al no devolver ningún resultado.

La sintaxis de una función:

**TipoDatoDev nombreFuncion(TipoDato1 parametro1, ..., TipoDatoN parametroN)**

TipoDatoDev indica el tipo de dato que devolverá la función cuando finalice su ejecución. Por otro lado, TipoDatoN es el tipo de dato que tendrá el parámetro N.

En la definición de una función (o de un procedimiento), estos parámetros son los denominados **formales**. Por otro lado, tenemos los **parámetros actuales** que son los valores concretos que se suministran a una función cuando es invocada.

Ejemplo:

Parámetros formales: `int calculaDato(int x, int y)`

Parámetros actuales: `valor = calculaDato(50, 70)`

La sintaxis de un procedimiento:

```
void nombreProcedimiento(TipoDato1 parametro1, ..., TipoDatoN parámetroN)
```

### Ámbito de una variable

Es la porción de código en que es visible esa variable. Una variable sólo puede utilizarse dentro de su ámbito. En función de su ámbito, las variables se clasifican en:

- **Variables GLOBALES:** son aquellas variables que se definen fuera de cualquier función/procedimiento del programa. Su ámbito es todo el código posterior a su declaración. Se pueden ver y usar en cualquier subrutina definida después.
- **Variables LOCALES:** son aquellas variables definidas dentro de una función/procedimiento. Su ámbito es todo el código posterior a su declaración dentro del cuerpo de la subrutina.

Por tanto, el ámbito de los parámetros definidos en la cabecera de una función/procedimiento, es el cuerpo de dicha subrutina. Lo mismo ocurre con las variables locales, sólo pueden verse y usarse dentro de la subrutina en la que están.

### Paso de parámetros

Existen dos formas de realizar el paso de parámetros en las funciones o procesos:

- **Paso de parámetros por VALOR:** en este caso el parámetro formal recibe una copia del valor que tenga el parámetro real. Puesto que el parámetro formal de la función/proceso trabajará con una copia del valor del parámetro real, es imposible que la acción pueda alterar el contenido de la variable real.
- **Paso de parámetros por REFERENCIA:** en este caso, el parámetro formal recibe una referencia del parámetro real, es decir, la dirección de memoria donde se almacena el valor del parámetro real. De esta forma, al disponer de la dirección de memoria, la función/proceso puede alterar el contenido de la variable que hace de parámetro real.

### Return (funciones)

La declaración **return** se usa para regresar explícitamente de un método. Es decir, hace que un control de programa se transfiera nuevamente a quién llama del método.

En una función el uso normal es para devolver el resultado que se ha obtenido con la ejecución del cuerpo de la función.

Ejemplo: Indique cuántas instrucciones del siguiente código son ejecutadas.

```
boolean t = true;
System.out.println("Antes de return.");

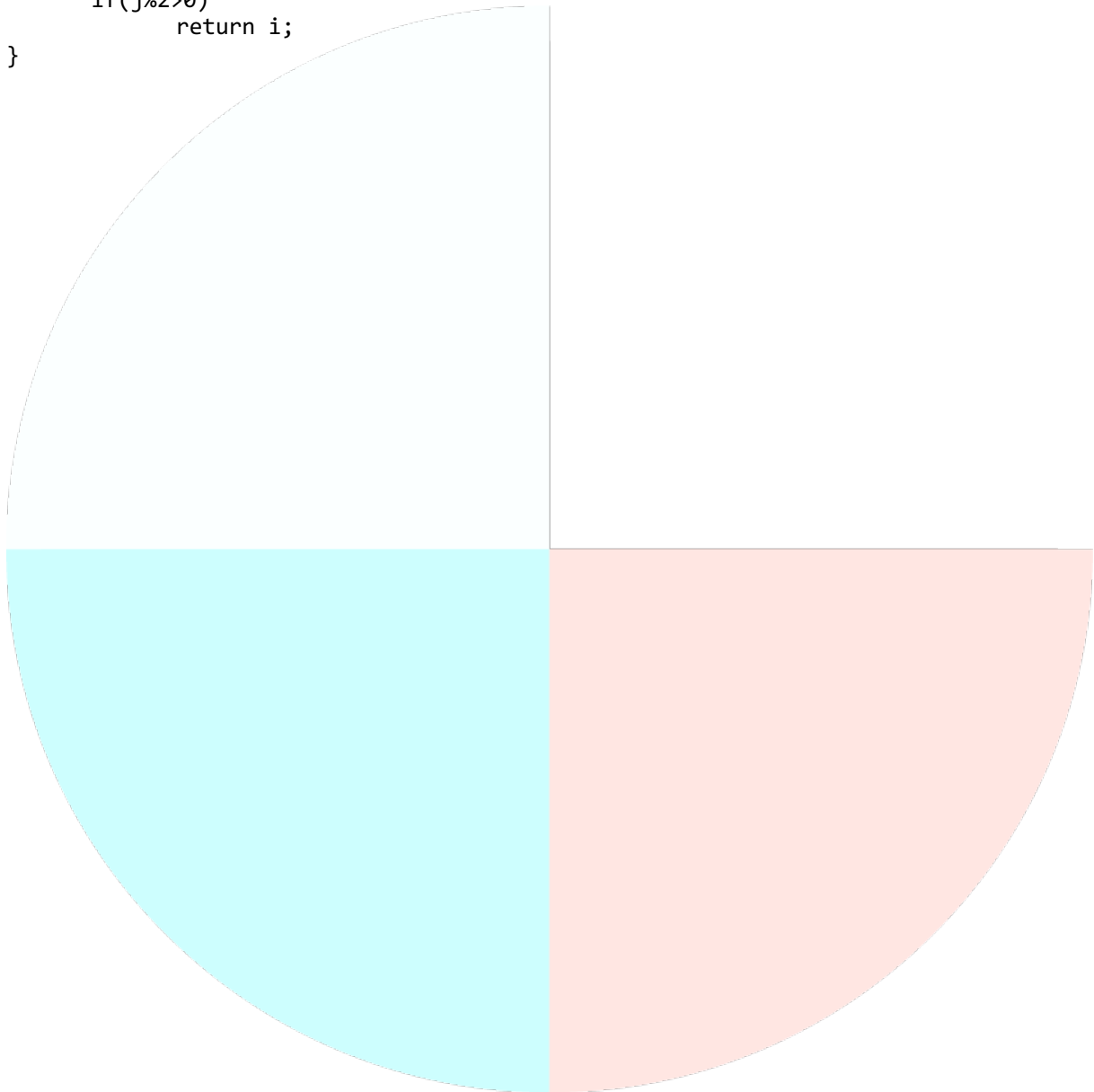
if (t)
    return;

System.out.println("Salida.");
```

Ejemplo: Indique cuántas veces se ejecutan los códigos que se especifican.

```
for (int i = 1; i <= 20; i += 2)
    int j = ++i;
    if(j%2>0)
        return i;
```

```
for (int i = 1; i <= 20; i += 2) {
    int j = ++i;
    if(j%2>0)
        return i;
}
```



## 7. VECTORES Y REGISTROS

Una **array** o **vector** es una colección de variables del mismo tipo, a la que se hace referencia por un nombre común. Los arrays pueden tener una o más dimensiones, aunque el **array unidimensional** es el más común.

La ventaja principal de un array es que organiza los datos de tal manera que puede ser manipulado fácilmente. Además, los arrays organizan los datos de tal manera que se pueden ordenar fácilmente.

Aunque los arrays en Java se pueden usar como matrices en otros lenguajes de programación, tienen un atributo especial: **se implementan como objetos**. Este hecho es una de las razones por las que la discusión de los arrays se pospuso hasta que se introdujeron los objetos. Al implementar arrays como objetos, se obtienen varias ventajas importantes, una de las cuales es que **los arrays no utilizados pueden ser recolectados**.

Un array unidimensional es una lista de variables relacionadas. Tales listas son comunes y frecuentes en la programación de aplicaciones.

La forma general de declarar un array unidimensional es:

**tipo nombre-array[];**

o

**tipo [] nombre-array;**

La declaración de un array tiene dos componentes: **el tipo y el nombre**.

- **Tipo:** declara el tipo de los elementos del array. El tipo de elemento determina el tipo de datos de cada elemento que comprende el vector. Al igual que la matriz de tipo *int*, también podemos crear una matriz de otros tipos de datos primitivos como *char*, *float*, *double*... o tipo de datos definido por el usuario (objetos de una clase). Por lo tanto, el tipo de elemento para el array determina el tipo de datos que el mismo contendrá.
- **Nombre:** denominación de la variable por la que se podrá acceder a los elementos de este.

Ejemplos:

```
int intArray[];  
int[] intArray2;
```

```
byte byteArray[];  
short shortArray[];  
boolean booleanArray[];  
long longArray[];  
float floatArray[];  
double doubleArray[];  
char charArray[];
```

```
MyClass myClassArray[];  
Object[] ao;  
Collection[] ca;
```

Cuando un array se declara, solo se crea una referencia del array. Para realmente crear o dar memoria al array, podemos crear un array de la siguiente manera:

```
nombre-array = new tipo [tamaño];
```

Ejemplos:

```
int intArray[]; //declarando un array
intArray = new int[20]; // asignando memoria al array
int[] intArray = new int[20]; // combinando ambas declaraciones en una
```

Los elementos en el vector asignado por new se inicializarán automáticamente a cero (para tipos numéricos), false (para booleano) o null (para tipos de referencia).

Obtener un array es un proceso de dos pasos. Primero, debe declarar una variable del tipo de array deseado. En segundo lugar, debe asignar la memoria que mantendrá el array, usar new y asignarla a la variable del array. Por lo tanto, todos los arrays se asignan dinámicamente.

Para acceder a cada elemento del array se hace a través de su índice. El índice comienza con 0 y termina en (tamaño total del array) - 1. Se puede acceder a todos los elementos de la matriz usando el bucle for en Java.

```
//acceder a los elementos del array
for (int i = 0; i < arr.length; i++)
System.out.println("Elemento en el índice " + i + " : "+ arr[i]);
```

Los arrays multidimensionales o matrices son **arrays de arrays**, donde cada elemento del array contiene la referencia de otro array. Se crea una matriz multidimensional al agregar un conjunto de corchetes ([ ]) por dimensión.

La forma general de declarar un array multidimensional es:

```
int[][] intArray = new int[10][20]; //un array 2D o matrix
int[][][] intArray = new int[10][20][10]; //un array 3D
```

En relación con los registros, ya fueron vistos en temas anteriores.

## 8. ESTRUCTURA DE UN PROGRAMA

Cualquier programa que se precie puede ser definido en 3 grandes bloques:

- **Entrada de datos**: código que se encarga de suministrar a la aplicación desarrollada de los datos necesarios para el cálculo computacional o tarea a realizar. Las fuentes pueden ser diversas: introducidos directamente por el usuario, remitidos por un periférico, extraídos de bases de datos u otras fuentes de datos, suministrados por otras aplicaciones, enviados por sensores...
- **Procesamiento de datos**: es el corazón del programa, el motivo por el cual fue desarrollado. Está constituido por el código que realiza todo el procesamiento de datos, incluidos los recogidos por la entrada de datos ya expuesta.
- **Salida de datos**: tras el procesamiento se produce un resultado que será guardado en base de datos, enviado a un periférico de salida, dirigido a un actuador, remitido a otro programa o simplemente mostrada por pantalla al usuario.

